

# Computer Networks Interview Study Guide

Nine topic areas, discovered from current interview-report and authoritative sources rather than a fixed syllabus — spanning the OSI/TCP-IP fundamentals, addressing and routing, TCP's reliability machinery, DNS, the HTTP/1.1→HTTP/3 evolution, TLS security, production infrastructure (load balancers, CDNs, VPCs), reliability engineering patterns, and a full packet-journey debugging toolkit. TCP and congestion-control claims are checked against the relevant RFCs directly.

70 questions • Basic → Intermediate → Advanced

9 topic areas

ASCII diagrams • dig/tcpdump/ss commands • packet journey

Sourced from GATE, GfG, Glassdoor, IETF RFCs, Cloudflare &amp; AWS docs

Updated Sep 2026

## WHO THIS IS FOR

Intern

Junior

Mid

Senior

- **Intern/Junior** — expect OSI/TCP-IP layers, TCP vs UDP, and the "what happens when you type a URL" question (Sections 1–5).
- **Mid-level** — expect you to read a tcpdump/ss trace and reason about retransmissions or a failed handshake (Sections 3, 9).
- **Senior/SRE/Network Engineer** — expect production infrastructure trade-offs and live debugging scenarios across Sections 6–9.

## WHAT INTERVIEWERS EVALUATE

- **Protocol understanding** — naming the actual mechanism (SYN/ACK, cwnd, TTL), not a vague gesture at "networking stuff."
- **Layered troubleshooting** — isolating a fault to physical/network/transport/application, not guessing.
- **Latency reasoning** — distinguishing latency from throughput, and knowing what actually moves each.
- **Correct terminology** — precise use of terms like idempotent, head-of-line blocking, forward secrecy.

## 01 Core Networking Fundamentals

The layered vocabulary every later section assumes. Interviewers use this section to confirm you can place any given protocol at the right layer before asking anything harder.

## 1.1

## BASIC

## CONCEPT

## What is the OSI model, and what does each layer do?

The OSI (Open Systems Interconnection) model is a seven-layer conceptual framework describing how data moves from an application on one machine to an application on another, with each layer providing a service to the layer above it and relying on the layer below.

| LAYER                  | RESPONSIBILITY                                | EXAMPLES                              |
|------------------------|-----------------------------------------------|---------------------------------------|
| <b>7. Application</b>  | Protocols applications speak directly         | HTTP, DNS, SMTP                       |
| <b>6. Presentation</b> | Data format, encryption, compression          | TLS, JPEG encoding                    |
| <b>5. Session</b>      | Establishes/manages/tears down sessions       | Session tokens, sockets API           |
| <b>4. Transport</b>    | End-to-end delivery, reliability, ports       | TCP, UDP                              |
| <b>3. Network</b>      | Logical addressing, routing between networks  | IP, ICMP, routers                     |
| <b>2. Data Link</b>    | Node-to-node delivery on one physical segment | Ethernet, MAC addresses, switches     |
| <b>1. Physical</b>     | Raw bits over a physical medium               | Cables, radio signals, voltage levels |

Note it's an academic, vendor-neutral reference model — real protocol stacks (below) don't map onto it cleanly, which is exactly why it's paired with the practical TCP/IP model next.

OSI 7 layers

layered service model

## TCP/IP model vs OSI model — and why did TCP/IP "win" in practice?

| OSI (7 LAYERS)                     | TCP/IP (4 LAYERS)     |
|------------------------------------|-----------------------|
| Application, Presentation, Session | Application           |
| Transport                          | Transport             |
| Network                            | Internet              |
| Data Link, Physical                | Network Access (Link) |

OSI was designed first, as a comprehensive theoretical standard; TCP/IP was designed second, as the pragmatic protocol suite that actually shipped and became the real Internet. TCP/IP "won" not because it's more elegant — it collapses OSI's Session and Presentation layers, which turned out not to need dedicated standalone protocols in practice — but because it was the suite that was actually implemented, deployed, and iterated on at Internet scale, while OSI's own protocol suite (as opposed to its reference model) never achieved comparable adoption. In interviews, OSI is used as the shared *teaching vocabulary* ("that's a Layer 3 problem"), while TCP/IP is the model that describes what's actually running.

TCP/IP 4-layer model

reference model vs deployed suite

## 1.3 BASIC CONCEPT What is encapsulation?

Encapsulation is the process of wrapping data with a new header (and sometimes trailer) at each layer as it moves down the stack for transmission — each layer treats everything handed to it by the layer above as opaque payload, and adds only the header it's responsible for. Decapsulation is the reverse, stripping headers off one by one as data moves up the stack at the receiver.

```
Application data:      [ HTTP request ]
Transport (TCP):       [ TCP hdr | HTTP request ] → "segment"
Network (IP):          [ IP hdr | TCP hdr | HTTP request ] → "packet"
Data Link (Eth): [Eth hdr | IP hdr | TCP hdr | HTTP req | Eth trlr] → "frame"
Physical:              01001011010... → "bits"
```

The name for the encapsulated unit changes at each layer — segment (Transport), packet (Network), frame (Data Link) — and interviewers do listen for this precise terminology rather than "packet" used loosely for all of them.

encapsulation

segment/packet/frame

## 1.4 INTERMEDIATE COMPARISON MAC address vs IP address — why do we need both?

A **MAC address** is a physical, hardware-burned Layer 2 identifier, unique to a network interface card, used only for delivery *within one local physical segment* (a switch doesn't understand IP addresses at all — it forwards frames based purely on MAC address tables). An **IP address** is a logical, Layer 3 identifier, assignable and reassignable by network administrators (or DHCP), used for routing *across* different networks — a router's whole job is reading the destination IP and deciding which next network segment to forward toward. The two are needed together because they solve different problems: MAC handles "who on this exact wire," IP handles "how do I get to a completely different network possibly thousands of hops away." **ARP** (2.3) is the protocol that bridges them — translating a known IP address into the MAC address needed to actually deliver a frame on the local segment.

MAC address

IP address

logical vs physical addressing

## 1.5 INTERMEDIATE COMPARISON Switch vs router vs hub.

|                            | HUB                             | SWITCH                   | ROUTER                                              |
|----------------------------|---------------------------------|--------------------------|-----------------------------------------------------|
| <b>Layer</b>               | Physical (1)                    | Data Link (2)            | Network (3)                                         |
| <b>Forwarding decision</b> | None — repeats to all ports     | MAC address table        | IP routing table                                    |
| <b>Collision domain</b>    | One shared domain for all ports | Separate domain per port | Separate domain per port                            |
| <b>Connects</b>            | Devices on one segment          | Devices on one LAN       | Different networks (LAN to LAN, or LAN to Internet) |

Hubs are essentially obsolete in modern networks (replaced entirely by switches), but the comparison remains a standard way interviewers confirm you understand *why* a switch is strictly better within one LAN — eliminating the shared collision domain a hub forces on every connected device.

collision domain

MAC table

routing table

## Scenario: "I can't reach a website." Walk through diagnosing it layer by layer.

1. **Physical/Data Link** — is the interface even up? (`ip link`) Is Wi-Fi connected, cable plugged in?
2. **Network** — does the machine have a valid IP address and default gateway (`ip addr`)? Can it reach the gateway and beyond (`ping 8.8.8.8`, bypassing DNS to isolate routing from name resolution)?
3. **DNS (Application-layer service, but logically checked here)** — does the domain resolve (`dig example.com`)? If ping-by-IP works but ping-by-name doesn't, the fault is isolated to DNS, not routing.
4. **Transport** — can a TCP connection actually be established on the target port (`curl -v` or `nc -zv host 443`)? A successful ping (ICMP) with a failed TCP connect points at a firewall/security-group rule blocking that specific port, not a routing problem (9.5 covers this exact gotcha in depth).
5. **Application** — if the TCP connection succeeds, is the actual HTTP/TLS exchange failing (bad certificate, wrong Host header, application error)? This is where `curl -v`'s verbose output becomes the primary diagnostic tool.

The methodology, not any single command, is what's being tested: work outward from Layer 1/2 (is there even a link) toward Layer 7 (is the application actually responding correctly), narrowing at each step.

layer-by-layer isolation

## Why doesn't a successful ping guarantee an HTTP request will succeed?

This is a direct consequence of 1.6's layered model, and one of the most common "gotcha" framings interviewers use: **ping uses ICMP**, a completely separate Network-layer protocol from TCP — it only proves that IP-layer routing between the two hosts works and that the destination host is up and responding to ICMP echo requests. It proves *nothing* about whether: the target port is open and something is actually listening on it (Transport layer), a firewall or security group is specifically blocking TCP/443 while still permitting ICMP (very common — ICMP and TCP are filtered independently), or the application server itself is healthy and correctly handling requests (Application layer). Each layer is an independent point of failure, and a healthy lower layer says nothing about the health of the layers above it — this is precisely why 9.5 dedicates a full debugging scenario to exactly this trap in production.

ICMP ≠ TCP reachability

independent layer failures

### REAL INTERVIEW SIGNAL

Glassdoor-reported network/SRE interviews repeatedly frame troubleshooting as an explicit walk through the OSI layers rather than a single "what's broken" guess — matching how 1.6 and 1.7 are structured here.

### WHAT THE WEB SOURCES EMPHASIZE

GeeksforGeeks' and Naukri's 2026 networking-interview material both foreground OSI/TCP-IP, IP addressing, routers vs switches, and NAT/DHCP as the recurring core-fundamentals cluster — directly shaping which comparisons this section leads with.

## 02 Addressing & the Network Layer

GATE's CN syllabus and industry sources converge tightly here: IP addressing/subnetting math, ARP/ICMP, NAT, and the routing-algorithm comparison that both academic and production interviews still test.

## 2.1 BASIC COMPARISON IPv4 vs IPv6.

|                | IPv4                                  | IPv6                                                            |
|----------------|---------------------------------------|-----------------------------------------------------------------|
| Address length | 32-bit (~4.3 billion addresses)       | 128-bit (effectively unlimited)                                 |
| Format         | Dotted decimal (192.168.1.1)          | Hex, colon-separated (2001:db8::1)                              |
| NAT need       | Widespread, due to address exhaustion | Not needed for address space reasons                            |
| Header         | Variable-length, includes checksum    | Fixed-length, no header checksum (relies on lower/upper layers) |

The interview-relevant framing: IPv6 exists specifically because IPv4's 32-bit space (~4.3 billion addresses) has been effectively exhausted against a far larger number of connected devices — NAT (2.4) was the practical stopgap that let IPv4 survive long past its original design capacity, which is also why IPv6 adoption has been slower than originally projected: NAT was "good enough" to defer the problem.

IPv4 exhaustion

IPv6

## 2.2 BASIC CONCEPT What is subnetting and CIDR notation?

Subnetting divides one larger IP address block into smaller sub-networks, by using a subset of the host bits to identify the subnet instead of individual hosts. **CIDR** (Classless Inter-Domain Routing) notation, `a.b.c.d/n`, states the address alongside how many leading bits are the fixed network prefix — the remaining  $32 - n$  bits identify hosts within that subnet.

`192.168.1.0/24`

↑ 24 network bits fixed, 8 host bits free → 256 addresses (254 usable)

`192.168.1.0/26`

↑ 26 network bits fixed, 6 host bits free → 64 addresses (62 usable)

Two usable addresses are always reserved per subnet: the all-zeros host bits (the network address itself) and the all-ones host bits (the broadcast address) — which is why a `/24` gives 254, not 256, usable host addresses.

CIDR notation

network address

broadcast address

---

## 2.3 BASIC CONCEPT What is ARP, and when is it used?

**Address Resolution Protocol** resolves a known IP address to the MAC address needed to actually deliver a frame on the local network segment (1.4). When a host needs to send a packet to an IP on its local subnet but doesn't yet know the corresponding MAC address, it broadcasts an ARP request ("who has 192.168.1.5?") to every device on the segment; the device owning that IP replies directly with its MAC address, and the sender caches the mapping (the ARP cache/table) to avoid repeating this for every subsequent packet. ARP only operates within one local segment — for a destination on a different network, a host instead ARPs for its *default gateway's* MAC address and hands the packet off to the router, which handles finding the next hop.

ARP   ARP cache   broadcast resolution

---

## 2.4 INTERMEDIATE CONCEPT What is NAT, and why does it exist?

**Network Address Translation** rewrites the source (or destination) IP address of packets as they cross a router boundary, most commonly allowing many devices on a private network (using private IP ranges like 192.168.x.x) to share one single public IP address for outbound Internet access — directly the practical response to IPv4 exhaustion (2.1). The router maintains a translation table mapping each internal (private IP, port) pair to a unique (public IP, port) pair for the duration of a connection, rewriting packets in both directions transparently.

**SNAT** (Source NAT) rewrites the source address — the common outbound case (many internal hosts sharing one public IP, as with an AWS NAT Gateway per the search results). **DNAT** (Destination NAT) rewrites the destination address — used for port forwarding and load balancing, directing inbound traffic destined for one public IP/port to a specific internal server.

NAT   SNAT   DNAT   private IP ranges

## 2.5 INTERMEDIATE CONCEPT What is ICMP, and what's it used for?

ICMP (Internet Control Message Protocol) is a Network-layer protocol used for diagnostic and error-reporting messages, not for carrying application data. Two of its most interview-relevant uses: **ping** sends ICMP Echo Request messages and listens for Echo Reply, testing basic reachability and round-trip latency (with the important limitation from 1.7 — it says nothing about TCP-layer reachability). **traceroute** exploits ICMP's "Time Exceeded" message: it sends packets with a steadily increasing **TTL** (Time To Live) value, and each router that decrements a packet's TTL to zero sends back an ICMP Time Exceeded message identifying itself — revealing the router-by-router path to the destination, one hop per TTL value. ICMP is also central to **Path MTU Discovery** (2.8) — a router that must fragment a packet it can't fragment (the Don't-Fragment bit is set) sends back an ICMP "Fragmentation Needed" message reporting the MTU it can actually support.

ICMP

TTL

traceroute mechanism

## 2.6 INTERMEDIATE COMPARISON Distance vector vs link state routing.

|                               | DISTANCE VECTOR                                             | LINK STATE                                                                                    |
|-------------------------------|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>What each router knows</b> | Only its neighbors' reported distances ("routing by rumor") | The full network topology (after flooding)                                                    |
| <b>Update method</b>          | Periodically shares its entire routing table with neighbors | Floods link-state info to every router; each computes shortest paths independently (Dijkstra) |
| <b>Convergence</b>            | Slower, prone to loops                                      | Faster, more robust                                                                           |
| <b>Known weakness</b>         | <b>Count-to-infinity</b> problem                            | Higher memory/CPU cost (full topology per router)                                             |
| <b>Examples</b>               | RIP                                                         | OSPF                                                                                          |

The **count-to-infinity** problem: if a link fails, distance-vector routers can end up incrementally increasing their advertised distance to the dead destination based on stale info from each other, converging only very slowly (or looping) instead of quickly recognizing the destination is unreachable. **Split horizon** (never advertising a route back to the neighbor it was learned from) is the standard mitigation.

distance vector

link state

count-to-infinity

split horizon

**Worked subnetting problem: split 192.168.1.0/24 into 4 equal subnets.**

Splitting a /24 into 4 subnets means "borrowing" 2 host bits to create 4 subnet identifiers ( $2^2 = 4$ ), moving the prefix from /24 to /26:

| SUBNET   | NETWORK ADDRESS  | USABLE RANGE         | BROADCAST     |
|----------|------------------|----------------------|---------------|
| <b>1</b> | 192.168.1.0/26   | 192.168.1.1 – .62    | 192.168.1.63  |
| <b>2</b> | 192.168.1.64/26  | 192.168.1.65 – .126  | 192.168.1.127 |
| <b>3</b> | 192.168.1.128/26 | 192.168.1.129 – .190 | 192.168.1.191 |
| <b>4</b> | 192.168.1.192/26 | 192.168.1.193 – .254 | 192.168.1.255 |

Each /26 has  $2^6 = 64$  total addresses, 62 usable (network + broadcast reserved, per 2.2). The pattern to internalize rather than memorize: borrowing  $n$  host bits multiplies the number of subnets by  $2^n$  and divides the addresses per subnet by the same factor — always a power-of-two trade between subnet count and subnet size.

subnet borrowing

/26 math

## Bulk transfers fail while pings succeed. Diagnose via MTU/fragmentation.

**MTU** (Maximum Transmission Unit) is the largest frame size a network link can carry — commonly 1500 bytes on Ethernet. **Path MTU Discovery (PMTUD)** lets a sender find the smallest MTU along an entire path by sending packets with the **Don't Fragment (DF)** bit set; if a router along the path has a smaller MTU than the packet, per 2.5 it replies with an ICMP "Fragmentation Needed" message specifying the MTU it can actually handle, and the sender shrinks its packets accordingly.

The specific, documented failure mode (per RFC 2923): small packets — like a ping's ICMP echo, or the initial bytes of an interactive session — fit within even a reduced MTU and succeed normally. Bulk transfers, however, send large packets that require fragmentation guidance from that ICMP message — and if any firewall along the path is blocking ICMP entirely (a common, overly aggressive security posture), the sender never receives the "Fragmentation Needed" reply, never learns to shrink its packets, and the large packets are silently dropped with no error at all — a **PMTUD black hole**. The connection doesn't fail cleanly; it just hangs and eventually times out, which is exactly why this is a notoriously hard-to-diagnose failure: everything about the connection *looks* fine (ping works, TCP handshake completes) until real data volume is sent. Diagnosis: `tracert` reports the discovered path MTU directly; `tcpdump` filtering for the DF flag and ICMP type 3 code 4 (Fragmentation Needed) confirms whether those ICMP replies are actually arriving back.

MTU Path MTU Discovery PMTUD black hole DF flag

### WHAT INTERVIEWERS LISTEN FOR

Naming this specific failure mode — ICMP blocked, so PMTUD silently fails, so only large packets hang — is a strong senior-level signal precisely because "ping works but it's still broken" is a genuinely confusing symptom without this mental model.

### WHAT THE WEB SOURCES EMPHASIZE

GATE's CN syllabus and prep material explicitly foreground distance-vector/link-state routing (including count-to-infinity), CIDR/subnetting arithmetic, and ARP/ICMP/NAT as the network-layer core — directly setting this section's scope and its emphasis on a worked subnetting problem.

## 03 Transport Layer: TCP, UDP & Reliability

GATE sources specifically flag TCP congestion control and sliding-window protocols as the transport layer's most heavily tested topics — this section is weighted accordingly, checked against RFC 5681/9293.

### 3.1 BASIC COMPARISON TCP vs UDP.

|                                | TCP                                           | UDP                                             |
|--------------------------------|-----------------------------------------------|-------------------------------------------------|
| <b>Connection</b>              | Connection-oriented (handshake required)      | Connectionless                                  |
| <b>Reliability</b>             | Guaranteed delivery, retransmission, ordering | Best-effort; no delivery/order guarantee        |
| <b>Overhead</b>                | Higher (headers, ACKs, state tracking)        | Minimal (small, stateless header)               |
| <b>Flow/congestion control</b> | Built in                                      | None (left to the application)                  |
| <b>Typical use</b>             | Web (HTTP/1.1, /2), email, file transfer      | DNS, video/voice streaming, gaming, QUIC/HTTP/3 |

connection-oriented

best-effort delivery

### 3.2 BASIC CONCEPT The TCP three-way handshake and four-way close.

```
Client                                Server
|----- SYN (seq=x) ----->|  client requests connection
|<----- SYN-ACK (seq=y, ack=x+1) ---|  server acknowledges + requests back
|----- ACK (ack=y+1) ----->|  client acknowledges - ESTABLISHED

...data flows...

|----- FIN ----->|  client done sending
|<----- ACK -----|
|<----- FIN -----|  server done sending
|----- ACK ----->|  client acks; enters TIME_WAIT (3.8)
```

The handshake's SYN → SYN-ACK → ACK exchange exists to do two things simultaneously: confirm both directions of the connection actually work, and let each side communicate its own **initial sequence number**, the base value used for reliably ordering and acknowledging every subsequent byte. The four-way close is asymmetric and independent in each direction — either side can initiate, and TCP allows "half-closed" connections where one side has finished sending but the other hasn't yet.

SYN/SYN-ACK/ACK

initial sequence number

FIN

---

### 3.3 BASIC CONCEPT What's the difference between a socket and a port?

A **port** is just a 16-bit number (0–65535) identifying a specific application/service on a host — how the transport layer knows which running process a given segment belongs to (port 443 for HTTPS, port 22 for SSH). A **socket** is the full combination that uniquely identifies one specific TCP connection: (source IP, source port, destination IP, destination port, protocol). This is why a server listening on port 443 can serve thousands of simultaneous client connections through that single port — each individual connection is a distinct socket, distinguished by its unique combination of client IP and client port, even though the destination IP and port (the server's) are identical across all of them.

port socket 5-tuple

---

### 3.4 INTERMEDIATE CONCEPT Walk through the key TCP states.

```
CLOSED → LISTEN           (server waiting for connections)
        → SYN_SENT        (client sent SYN, awaiting SYN-ACK)
        → SYN_RECEIVED    (server got SYN, sent SYN-ACK)
        → ESTABLISHED     (handshake done, data can flow)
        → FIN_WAIT_1/2    (initiating the close)
        → TIME_WAIT       (waiting to ensure clean close – see 3.8)
        → CLOSED
```

Most interview questions about TCP state focus on three: **LISTEN** (a server socket bound and ready, no connection yet — this is what `ss -tlnp` shows), **ESTABLISHED** (an active, data-flowing connection), and **TIME\_WAIT** (3.8's dedicated question) — because these three are what you actually see and reason about when debugging a live system.

TCP state machine LISTEN ESTABLISHED

## Flow control vs congestion control — why are they different problems?

**Flow control** protects the *receiver* — it prevents a fast sender from overwhelming a slow receiver's buffer, using the receiver-advertised **window size** in every ACK to tell the sender exactly how much unacknowledged data it's currently willing to buffer. **Congestion control** protects the *network* itself — it prevents a sender from overwhelming the links and routers between sender and receiver, inferred indirectly (classic TCP has no direct signal from routers) via observed packet loss or delay, using the sender's own **congestion window (cwnd)**. The two are genuinely independent constraints, and the sender is only ever permitted to have in flight the *minimum* of the two windows — a receiver with plenty of buffer space doesn't help if the network path itself is congested, and vice versa.

flow control

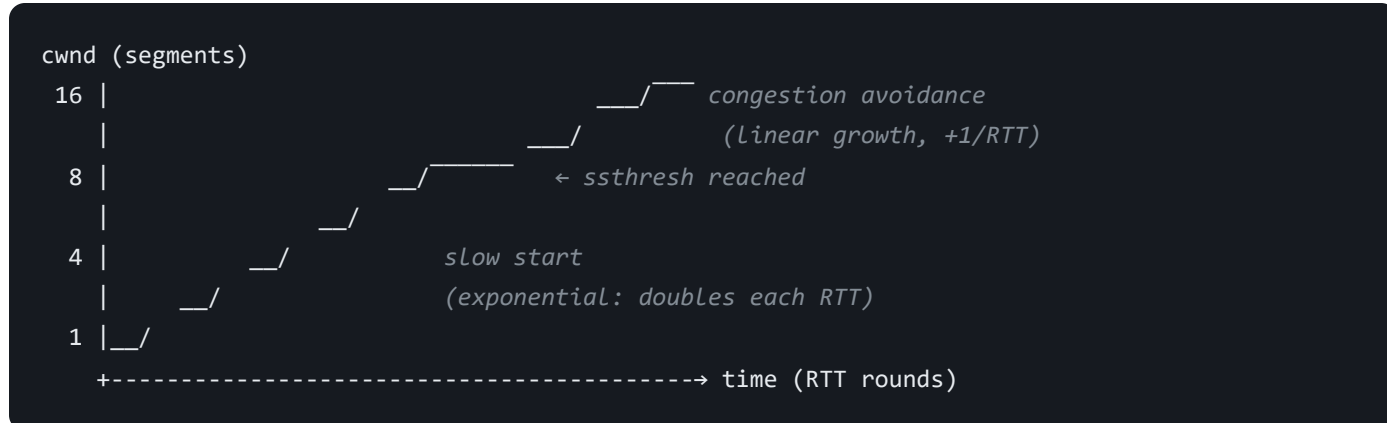
congestion control

receiver window

cwnd

### 3.6 INTERMEDIATE CONCURRENCY Explain TCP congestion control's phases, per RFC 5681.

Per RFC 5681, TCP congestion control comprises four intertwined algorithms:



- **Slow start** — cwnd begins small (historically 1-few segments, larger initial windows are now common) and *doubles* every round-trip as ACKs confirm successful delivery — deliberately fast, to quickly find roughly how much capacity is available, up to a threshold (**ssthresh**).
- **Congestion avoidance** — once past ssthresh, growth switches to linear (roughly +1 segment per RTT) — probing for more capacity much more cautiously, since the sender is now in territory where it previously observed loss.
- **Fast retransmit** — instead of waiting for a full retransmission timeout, receiving three duplicate ACKs (a strong signal a specific segment was lost, since later segments are arriving fine) triggers immediate retransmission of the missing segment.
- **Fast recovery** — after a fast retransmit, cwnd is cut (typically halved) rather than reset all the way back to slow start's minimum — a lighter penalty than a full timeout, since duplicate ACKs indicate the network is still delivering most traffic successfully.

slow start

congestion avoidance

ssthresh

fast retransmit

## A connection shows many TCP retransmissions. Diagnose it.

- **Real packet loss on the path** — congested links, a flaky wireless hop, or a failing piece of hardware. `mtr` (a continuous traceroute) run over time can reveal loss concentrated at a specific hop.
- **MTU/PMTUD black hole** (2.8) — large segments silently dropped, presenting as retransmissions of specifically large packets while small ones succeed.
- **Receiver overload** — if the receiver's application isn't reading fast enough, its advertised window (3.5) shrinks toward zero, and the sender may end up retransmitting probes; distinguishable from network loss because the advertised window itself is visibly shrinking in the capture.
- **An overly aggressive middlebox** (firewall, load balancer) resetting or dropping connections under its own load or idle-timeout policy.

```
# Confirm retransmissions are actually happening, and how many
$ ss -ti # look for "retrans:" in the per-connection info

# Capture and filter for TCP retransmissions specifically
$ tcpdump -i eth0 'tcp[tcpflags] & (tcp-syn|tcp-ack) != 0' -w cap.pcap
# then inspect in Wireshark, filter: tcp.analysis.retransmission
```

The diagnostic order that separates a strong answer: confirm the retransmissions are real (`ss -ti`) before reaching for a full packet capture, then use `tcpdump`/Wireshark's built-in retransmission analysis rather than eyeballing raw packets.

```
ss -ti      tcpdump retransmission filter
```

## What is `TIME_WAIT`, and why can too many of them cause production issues?

After the side that initiates a close sends its final ACK, it enters `TIME_WAIT` for a period (historically  $2 \times \text{MSL}$ , Maximum Segment Lifetime — commonly a minute or more) rather than immediately freeing the connection's resources. This exists for two real reasons: ensuring the final ACK is retransmitted if lost (so the other side's FIN can be fully, correctly acknowledged even after packet loss), and preventing delayed, stray packets from a now-closed connection from being misinterpreted as belonging to a brand-new connection that happens to reuse the same socket 5-tuple.

In production, a server that opens huge numbers of short-lived outbound connections (a common pattern in poorly-designed proxy or backend-calling-backend code) can accumulate enough sockets stuck in `TIME_WAIT` to exhaust the available **ephemeral port range** — new outbound connections start failing with "address already in use" or similar errors, even though the machine has plenty of CPU/memory headroom. The standard fixes: reuse persistent connections (connection pooling) instead of opening a new one per request, and where appropriate, tune socket options like `SO_REUSEADDR`/kernel `TIME_WAIT` reuse settings — never simply "shortening" `TIME_WAIT` carelessly, since that reintroduces the exact correctness risk the state exists to prevent.

`TIME_WAIT`

ephemeral port exhaustion

connection pooling

### REAL INTERVIEW SIGNAL

Linux-networking-for-SRE material found in search specifically calls out `TIME_WAIT` exhaustion as a real, recurring production incident pattern, not a theoretical edge case — reinforcing why this is framed as a debug question rather than a pure definition.

## UDP is "unreliable" — so how can QUIC (built on UDP) be a reliable protocol?

The gotcha hinges on a precise distinction: "unreliable" describes what UDP itself *guarantees* (nothing — no delivery confirmation, no ordering, no retransmission), not a hard limit on what can be *built on top of* UDP. UDP is deliberately minimal specifically so that something else can implement exactly the reliability semantics it actually needs, rather than being stuck with TCP's specific, fixed choices. QUIC (the transport underlying HTTP/3, Section 5) is exactly this: it runs over UDP but implements its own acknowledgment, retransmission, and congestion control logic in user space — reliable delivery, per stream, is a QUIC-layer guarantee, not a UDP-layer one. This is also precisely why QUIC can solve TCP's head-of-line blocking (5.4/5.5): because it isn't bound to TCP's single strictly-ordered byte stream, it's free to implement independent reliability per logical stream.

UDP as a minimal substrate

QUIC implements reliability in user space

### WHAT THE WEB SOURCES EMPHASIZE

GATE prep sources explicitly rank TCP congestion control and sliding-window protocols as the single most heavily tested transport-layer topic, while Glassdoor/SRE material independently converges on TIME\_WAIT and retransmission debugging as the practical, production-facing counterpart — both directly shaping this section's split between RFC-grounded theory and live debugging.

## 04 DNS & Name Resolution

The system that turns the "what happens when you type a URL" interview classic (Section 9) into something explainable in precise, layered detail.

#### 4.1 BASIC CONCEPT What is DNS, and what does the resolution chain look like?

DNS (Domain Name System) is the distributed, hierarchical system that translates human-readable domain names into IP addresses — without it, every request would require memorizing raw IP addresses.

```
Browser cache → OS cache → Recursive resolver (ISP or 1.1.1.1/8.8.8.8)
                                |
                                ├── queries a Root server ("where's .com?")
                                │   → returns the .com TLD server
                                ├── queries the TLD server ("where's example.com?")
                                │   → returns example.com's authoritative NS
                                ├── queries the Authoritative server
                                │   → returns the actual A/AAAA record
                                |
                                (result cached and returned to the client, per its TTL)
```

Each cache checked along the way (browser, OS, resolver) exists purely to avoid repeating this multi-hop lookup for every request — the full chain only runs on a genuine cache miss at every level.

recursive resolver root server TLD server authoritative server

#### 4.2 BASIC COMPARISON Common DNS record types.

| RECORD       | PURPOSE                                                                         |
|--------------|---------------------------------------------------------------------------------|
| <b>A</b>     | Domain name → IPv4 address                                                      |
| <b>AAAA</b>  | Domain name → IPv6 address                                                      |
| <b>CNAME</b> | An alias pointing to another domain name (not directly an IP)                   |
| <b>MX</b>    | Mail server responsible for the domain, with priority                           |
| <b>TXT</b>   | Arbitrary text — commonly used for domain verification, SPF/DKIM email security |
| <b>NS</b>    | The authoritative name servers for the domain                                   |

A/AAAA CNAME MX NS

---

#### 4.3 INTERMEDIATE CONCEPT **DNS caching and TTL — the trade-off of long vs short TTL.**

Every DNS record is published with a **TTL** (Time To Live, in seconds) telling every resolver and client how long they're permitted to cache and reuse that answer before re-querying. A **long TTL** reduces DNS query volume and lookup latency for end users (more cache hits), but means a change to the record — updating an IP after migrating servers, say — takes correspondingly longer to actually propagate everywhere, since every resolver holding a still-valid cached copy has no reason to re-check. A **short TTL** allows fast propagation of changes, at the cost of more frequent lookups (added latency and resolver/server load). The standard production pattern: run with a normal, longer TTL, and deliberately lower it in advance of a planned change (a migration, a DNS-based failover) so the old, cached answers age out quickly once the actual change happens — directly setting up 4.7's scenario.

TTL

propagation delay

---

#### 4.4 INTERMEDIATE COMPARISON **Recursive vs iterative DNS query.**

In a **recursive** query, the client asks one server (the recursive resolver) for a final answer, and that resolver takes full responsibility for doing whatever additional querying is needed — chasing root → TLD → authoritative on the client's behalf — and returns only the complete, final result. In an **iterative** query, each server contacted returns either the final answer or a referral to the next server to ask, and the querier (typically the recursive resolver itself, in its own queries toward root/TLD/authoritative servers) does the chasing step by step. In practice, the client-to-resolver hop is recursive (the client wants a simple, complete answer), while the resolver-to-root/TLD/authoritative hops are iterative (each of those servers just says "ask them next," rather than doing the client's full job for it) — the two query styles operate at different hops of the same overall chain in 4.1's diagram.

recursive query

iterative query

referral

## A site is unreachable — use dig to isolate whether it's a DNS problem.

```
# Basic resolution — does the domain resolve at all, and to what?
$ dig example.com +short
93.184.216.34

# Query a specific, known-good public resolver directly, bypassing local/ISP cache
$ dig @8.8.8.8 example.com

# Query the authoritative server directly, bypassing every cache in the chain
$ dig @ns1.example.com example.com +norecurse

# Check the record's TTL explicitly (relevant to 4.3 and 4.7)
$ dig example.com | grep -A1 "ANSWER SECTION"
```

The diagnostic logic: if `dig @8.8.8.8` returns a correct, current answer but the browser/application still fails, the problem is *not* DNS at all — it's downstream, likely local cache staleness, routing, or the application layer (loop back to 1.6). If `dig` itself times out or returns `SERVFAIL/NXDOMAIN`, the fault is genuinely DNS — and querying the authoritative server directly (bypassing every recursive resolver and cache) isolates whether the record itself is actually correctly published, versus a caching/propagation issue somewhere in the resolver chain.

`dig +short``dig @resolver``SERVFAIL/NXDOMAIN`

## DNS-based load balancing / GeoDNS — main weakness vs an L4/L7 load balancer?

GeoDNS/DNS-based load balancing returns different IP addresses for the same domain depending on the querying resolver's location or configured health status — a coarse, cheap way to distribute traffic across regions or route around a fully dead datacenter. Its fundamental weakness, directly tied to 4.3: DNS answers are **cached**, at every layer of 4.1's chain, for the duration of their TTL — DNS-based failover cannot react faster than the record's TTL allows, and worse, has no visibility into or control over individual client behavior once an answer has been cached and handed out; a client holding a stale cached IP for a now-dead server will keep trying it until its own cache expires, regardless of what the DNS record now says. A real Layer 4/Layer 7 load balancer (Section 7), by contrast, sits directly in the request path, makes a fresh routing decision on every single connection or request, and can react to a backend failure within seconds via active health checks — a fundamentally faster and more precise mechanism than anything DNS-level can offer, which is why DNS-based routing is typically used for coarse, regional-scale distribution, layered on top of real load balancers doing the fine-grained work within each region.

GeoDNS

DNS caching limits failover speed

## You updated a DNS record, but users still hit the old IP for hours. Why?

This is 4.3's TTL trade-off manifesting as an incident. The old record was published with some TTL — say, 4 hours — and every resolver and client that cached it *before* the change has no reason to re-query until that cached copy's TTL expires, regardless of when the authoritative record actually changed; DNS provides no mechanism to proactively "push" an update out to already-caching resolvers. Some resolvers (particularly certain ISPs) are also documented to not strictly respect published TTLs, caching longer than instructed, which compounds the delay unpredictably. What should have been done differently, tying directly back to 4.3: lower the record's TTL well *in advance* of the planned change — giving every existing cached copy across the Internet time to naturally expire on the old, short TTL before the actual IP change is made — then raise the TTL back to normal once the migration is confirmed stable. Making the change first and only then noticing propagation is slow is a scheduling mistake, not a DNS malfunction.

TTL not respected retroactively

pre-lower TTL before migration

#### WHAT THE WEB SOURCES EMPHASIZE

Both the "what happens when you type a URL" sources and SRE-focused search results converge on `dig` (not `nslookup`) as the modern standard diagnostic tool, and independently emphasize the resolver-cache chain as the part of DNS most often misunderstood in interviews — directly shaping 4.1 and 4.5's structure.

## 05 HTTP & the Web Stack

The HTTP/1.1 → HTTP/2 → HTTP/3 evolution is one of the most current, fast-moving parts of this guide — grounded directly in Cloudflare's own engineering writing on QUIC.

### 5.1 BASIC CONCEPT

#### What is HTTP, and what are the common methods/status code classes?

HTTP (HyperText Transfer Protocol) is the Application-layer protocol underlying essentially all Web communication — a request/response protocol where a client sends a request (method, path, headers, optional body) and a server returns a response (status code, headers, optional body).

| STATUS CLASS | MEANING                                                           |
|--------------|-------------------------------------------------------------------|
| <b>2xx</b>   | Success (200 OK, 201 Created, 204 No Content)                     |
| <b>3xx</b>   | Redirection (301 Moved Permanently, 304 Not Modified)             |
| <b>4xx</b>   | Client error (400 Bad Request, 401 Unauthorized, 404 Not Found)   |
| <b>5xx</b>   | Server error (500 Internal Server Error, 503 Service Unavailable) |

Common methods: GET (retrieve, no body), POST (create/submit, has a body), PUT (replace a resource, has a body), DELETE, PATCH (partial update).

status code classes

request/response

## 5.2 BASIC COMPARISON PUT vs POST — the idempotency angle.

**Idempotent** means calling the operation multiple times with the same input produces the same end state as calling it once — no additional side effect from repetition. PUT is defined to be idempotent: `PUT /users/5 {"name": "Alice"}` called five times in a row leaves user 5 with the name "Alice" exactly once, in exactly the same state as calling it once — it replaces the resource wholesale. POST is *not* idempotent by definition: `POST /users {"name": "Alice"}` called five times typically creates five separate new users, since it's semantically "create a new thing," and calling it again naturally creates another. This distinction is directly load-bearing for Section 8's reliability patterns: it's generally safe to blindly retry a failed PUT, but retrying a failed POST without an idempotency key (8.6) risks duplicate side effects — duplicate orders, duplicate charges.

idempotent

PUT replaces, POST creates

## 5.3 INTERMEDIATE COMPARISON HTTP/1.1 vs HTTP/2 vs HTTP/3.

|                                | HTTP/1.1                                                 | HTTP/2                                                   | HTTP/3                                                 |
|--------------------------------|----------------------------------------------------------|----------------------------------------------------------|--------------------------------------------------------|
| <b>Transport</b>               | TCP                                                      | TCP                                                      | QUIC (over UDP)                                        |
| <b>Requests per connection</b> | One at a time per connection (or pipelined, rarely used) | Multiplexed — many concurrent streams, one connection    | Multiplexed — many independent streams, one connection |
| <b>Head-of-line blocking</b>   | Yes, at the application level                            | Fixed at HTTP level, but reintroduced at TCP level (5.4) | Solved at the transport level (5.5)                    |
| <b>Header compression</b>      | None (headers sent as plain text every request)          | HPACK                                                    | QPACK                                                  |
| <b>Handshake cost</b>          | TCP + separate TLS handshake                             | TCP + separate TLS handshake                             | Combined transport+crypto handshake, 0-1 RTT (5.8)     |

multiplexing

HPACK/QPACK

## What is head-of-line blocking, and why did HTTP/2 only partially fix it?

**Head-of-line (HOL) blocking** is when one stalled item at the front of a queue blocks everything behind it, even items that are individually ready to proceed. In HTTP/1.1, this happens at the application level: without multiplexing, a slow response genuinely blocks subsequent requests on that same connection (worked around in practice by browsers opening several parallel TCP connections to the same server — itself a real cost in memory/CPU on both ends). HTTP/2's headline feature, **multiplexing**, interleaves many logical streams over one single TCP connection, fixing the application-level version of the problem directly. But — and this is the specific nuance interviewers listen for — HTTP/2 still runs over TCP, and TCP guarantees one strictly-ordered byte stream: if a single packet is lost, TCP withholds *every* byte that arrived after it, across *every* multiplexed HTTP/2 stream, until that one lost packet is retransmitted — so the head-of-line blocking problem didn't disappear, it just moved from the HTTP layer down to the TCP layer, and on a lossy connection (mobile networks with 1-5% loss are the documented example) HTTP/2 can actually perform *worse* than HTTP/1.1's multiple parallel connections, since a single loss now stalls every stream at once instead of just one connection's worth.

head-of-line blocking

TCP-level HOL blocking

### WHAT INTERVIEWERS LISTEN FOR

"HOL blocking moved from the HTTP layer to the TCP layer, it didn't disappear" is the precise, current-knowledge answer — many candidates stop at "HTTP/2 fixed head-of-line blocking," which is the outdated, incomplete version.

## What is QUIC, and how does HTTP/3 finally solve transport-level HOL blocking?

QUIC is a transport protocol built on top of UDP (3.9) rather than TCP, designed specifically to give HTTP/3 the multiplexing benefits of HTTP/2 without inheriting TCP's single-ordered-byte-stream constraint. QUIC implements reliability and ordering *per individual stream*, not across the whole connection — so when a packet belonging to one stream is lost, QUIC only needs to hold back and retransmit data for *that specific stream*; every other stream's data continues to be delivered to the application immediately, completely unaffected. This directly and structurally solves the problem 5.4 describes, rather than working around it — per Cloudflare's own published findings, this is specifically why HTTP/3 shows the largest real-world gains on lossy, high-latency links like mobile networks, exactly the condition where HTTP/2 could underperform HTTP/1.1.

QUIC

per-stream reliability

## 5.6

## ADVANCED

SQL

## Show an HTTP caching flow with real request/response headers.

```
// First request – no cached copy yet
GET /app.js HTTP/1.1
Host: example.com

// Response – server tells the client how long it can cache, and gives a validator
HTTP/1.1 200 OK
Cache-Control: max-age=3600
ETag: "a1b2c3d4"

// Second request, after max-age has expired – client must revalidate
GET /app.js HTTP/1.1
Host: example.com
If-None-Match: "a1b2c3d4"

// If the file hasn't actually changed server-side – cheap, no body sent
HTTP/1.1 304 Not Modified
```

Cache-Control: max-age lets the client skip contacting the server entirely until it expires — the fastest possible case. ETag, an opaque fingerprint of the resource's content, enables cheap revalidation afterward: the client sends it back via If-None-Match, and if the content hasn't changed, the server can reply 304 Not Modified with an empty body instead of re-sending the entire file — confirming the cached copy is still valid at a fraction of the bandwidth cost of a fresh 200.

Cache-Control

ETag

304 Not Modified

## Enabling HTTP/2 made performance worse on a lossy mobile network. Explain.

This is 5.4's TCP-level HOL blocking manifesting as a real, measured regression rather than a theoretical caveat. Before the change, the client was likely using several separate HTTP/1.1 TCP connections in parallel — each connection has its own independent congestion window and its own independent exposure to loss, so a lost packet on one connection only stalls the requests riding on *that* connection; the others continue delivering fine. After enabling HTTP/2, all requests were consolidated onto a *single* TCP connection for the efficiency of multiplexing — but that single connection now has a single shared exposure to loss: on a network with 1-5% packet loss (typical for mobile), losses happen often enough, and now every single one of them stalls every in-flight request across the entire page load simultaneously, not just a fraction of them. The fix that actually addresses the root cause, rather than reverting the HTTP/2 upgrade: move to HTTP/3/QUIC (5.5), which gets multiplexing's efficiency without reintroducing this specific shared-connection loss exposure — precisely the improvement path Cloudflare's own engineering blog documents for exactly this network profile.

single shared connection loss exposure

## What is 0-RTT resumption, and what security trade-off does it introduce?

A normal TLS 1.3 handshake (Section 6.2) requires at least one round trip before the client can send any encrypted application data. **0-RTT resumption** lets a client that has previously connected to a server send encrypted application data *immediately*, in its very first flight of packets, using a resumption key derived from the earlier session — QUIC's combined transport+crypto handshake (per Cloudflare's own documentation) is specifically designed to exploit this, reducing round trips from TCP+TLS's typical 2-3 down to 0-1 for a repeat connection to a known server, a real, measurable latency win especially on high-latency mobile links.

The trade-off: 0-RTT data is genuinely vulnerable to **replay attacks** — because it doesn't require a fresh round trip to prove the client is live and not just recording and resending an old, captured packet, an attacker who intercepts a 0-RTT request can potentially replay it to the server, and the server has no built-in way to distinguish a replayed copy from the original. This is why 0-RTT is only recommended for *idempotent* requests (5.2's concept resurfacing directly) — a replayed GET is harmless, but a replayed non-idempotent POST (a payment, an order) could cause real, duplicated damage, which is exactly why production systems that enable 0-RTT must explicitly restrict it to safe, idempotent operations.

0-RTT    replay attack    idempotency constrains 0-RTT use

#### WHAT THE WEB SOURCES EMPHASIZE

Cloudflare's engineering blog was the clearest, most current authority found on QUIC/HTTP/3 — specifically the finding that TCP-level HOL blocking is what HTTP/2 "moved rather than fixed," and that mobile/lossy-network performance is where HTTP/3's real-world gains concentrate, both used directly in 5.4/5.5/5.7 rather than paraphrased secondhand.

## 06 TLS & Network Security

Security questions test whether "it's encrypted" is the whole answer a candidate has, or the starting point for a much more precise one.

### 6.1 BASIC CONCEPT

#### What is TLS, and why use both asymmetric and symmetric encryption in one handshake?

TLS (Transport Layer Security, the successor to SSL) provides encryption, integrity, and authentication for data in transit — it's what turns HTTP into HTTPS. It uses **asymmetric encryption** (a public/private key pair) only during the handshake itself, to authenticate the server and safely agree on a shared secret — because asymmetric operations are computationally expensive. It then switches to **symmetric encryption** (one shared key, known to both sides) for the actual bulk data transfer, because symmetric algorithms are dramatically faster. This hybrid approach captures the best of both: asymmetric crypto's ability to establish trust and a shared secret between two parties who've never met before, without either side ever having to transmit a secret key in the clear, combined with symmetric crypto's speed for the actual, potentially large, volume of real traffic that follows.

asymmetric handshake

symmetric bulk encryption

## 6.2 BASIC CONCEPT Walk through the TLS 1.3 handshake.

```
Client                                     Server
|---- ClientHello (TLS version, cipher  ----->|
|      suites offered, client random,      |
|      key share) ----->|
|                                           |
|<--- ServerHello (chosen cipher suite,  -----|
|      server random, key share,          |
|      Certificate, Finished) -----|
|                                           |
|---- Finished ----->|
|                                           |
|===== application data, encrypted =====|
```

TLS 1.3 (the current standard) compressed the handshake to effectively one round trip by having the server send its key share, certificate, and a "Finished" message all in its very first reply — a meaningful improvement over TLS 1.2's multi-round-trip negotiation, and the direct basis for the 0-RTT resumption discussed in 5.8 for repeat connections.

ClientHello   ServerHello   TLS 1.3 1-RTT handshake

## 6.3 BASIC CONCEPT What is a certificate, and what is a certificate chain?

A TLS certificate is a digitally signed document (X.509 format) binding a public key to an identity — a domain name — issued by a **Certificate Authority (CA)**. It contains the domain(s) it's valid for, the server's public key, an expiration date, and the issuing CA's digital signature. Because browsers can't practically pre-trust every individual website's certificate directly, trust is delegated through a **chain**: the server's certificate (leaf) is signed by an intermediate CA certificate, which is itself signed by a root CA certificate that ships pre-trusted in the operating system/browser's trust store. Verification walks this chain from leaf up to a trusted root — if every signature in the chain checks out and the root is genuinely trusted, the leaf certificate (and thus the server presenting it) is considered authentic.

X.509 certificate   Certificate Authority   chain of trust   root store

## What is a MITM attack, and how does TLS certificate validation prevent it?

A Man-in-the-Middle attack positions an attacker between client and server, intercepting (and potentially altering) traffic while impersonating each side to the other's satisfaction. Encryption alone doesn't prevent this — if a client would accept *any* encryption key without verifying who it actually belongs to, an attacker could simply hand the client the attacker's own key, decrypt and read everything, then re-encrypt and forward it to the real server, with neither side any the wiser. This is exactly what certificate validation (6.3) is for: **authentication**, not just encryption — the client checks that the presented certificate is signed by a trusted CA chain *and* that it's actually issued for the specific domain being connected to. An attacker without the real server's private key cannot produce a certificate that both matches the domain and validates against a trusted root, so a client performing genuine, complete certificate validation will reject the connection rather than silently accepting the attacker's substitute — which is precisely why disabling or bypassing certificate validation (a common quick "fix" for a dev-environment TLS error) is such a serious, MITM-enabling mistake if it ever leaks into production.

MITM

authentication vs encryption

## 6.5 INTERMEDIATE SECURITY What is forward secrecy, and why does it matter?

**Forward secrecy** (or perfect forward secrecy) means that even if an attacker later obtains a server's long-term private key, they still cannot decrypt *past* recorded sessions. This is achieved by deriving each session's actual symmetric encryption key through an ephemeral key exchange (Diffie-Hellman, generating a fresh, temporary key pair for each individual session) rather than directly from the server's long-term key — once a session ends, the ephemeral keys involved are discarded and never stored, so there is nothing left for a later key compromise to unlock. Without forward secrecy — as in older RSA key exchange, per the search results — the session key is derived directly from the server's long-term private key, meaning an attacker who silently records encrypted traffic today and compromises the server's private key at any point in the future (even years later) can retroactively decrypt every session ever recorded. This is why forward secrecy is now considered a baseline requirement, not an optional hardening step, for modern TLS deployments — it changes a single future key compromise from "every past conversation is now readable" to "only future conversations are at risk, past ones remain safe."

forward secrecy

ephemeral Diffie-Hellman

retroactive decryption risk

---

## 6.6 INTERMEDIATE DESIGN What is TLS termination, and where does it typically happen?

TLS termination is the point where encrypted TLS traffic is decrypted back into plaintext — the server (or load balancer) at that point "terminates" the TLS connection. In a typical production architecture, TLS is terminated at the edge — a load balancer, CDN, or dedicated reverse proxy — rather than at each individual backend application server, for practical reasons: certificate management is centralized in one place instead of duplicated across every backend instance, the CPU cost of TLS (real, though modern hardware offload has shrunk it) is paid once at the edge rather than repeatedly by every backend, and the edge layer is exactly where features like WAF inspection or request routing need visibility into the actual plaintext request content anyway. Traffic from the termination point onward to backend servers may then travel as plain HTTP (a real trade-off examined directly in 6.8) or be re-encrypted with a second, typically simpler, internal TLS layer.

TLS termination

edge decryption

---

## 6.7 ADVANCED DEBUG Diagnose a reported "SSL handshake failed" error.

- **Expired certificate** — the single most common cause; check the certificate's validity window directly (`openssl s_client -connect host:443 | openssl x509 -noout -dates`).
- **Missing intermediate certificate** — the server sent only its leaf certificate without the intermediate CA cert needed to complete the chain (6.3); some clients/browsers that happen to already have the intermediate cached will succeed while others fail, producing confusing, inconsistent reports.
- **Self-signed or internally-issued certificate in production** — valid for internal testing, but any client without that specific issuer explicitly added to its trust store will correctly reject it as untrusted.
- **Cipher suite / TLS version mismatch** — an old client that only supports TLS 1.1 talking to a server hardened to require TLS 1.2+ (a legitimate, deliberate security posture) will fail to negotiate a common protocol version at all.
- **Hostname mismatch** — the certificate is valid and trusted, but issued for a different domain/subdomain than the one actually being requested (e.g. missing a wildcard, or a Subject Alternative Name gap).

The fastest real diagnostic: `openssl s_client -connect example.com:443` shows the full handshake attempt, the certificate chain actually presented, and the specific negotiated (or failed) protocol/cipher — almost always enough to immediately identify which of the above it is.

`openssl s_client`

chain/hostname/expiry/version mismatch

## TLS is terminated at the load balancer, then plaintext HTTP goes to backend servers inside the VPC. Acceptable?

This is a genuinely common, defensible production pattern — *if* the risk model actually supports it. The argument for acceptability: traffic between the load balancer and backend servers stays entirely within a private network (a VPC, Section 7.4) that's itself isolated from the public Internet via security groups/firewall rules, so the plaintext is never exposed to an external attacker in the way public-Internet traffic would be — the load balancer has already done the expensive, security-critical work of authenticating the client and encrypting the public-facing hop, which is where the real MITM risk (6.4) actually lives.

The argument against, and where this genuinely becomes unacceptable: it assumes the internal network is fully trusted, which is precisely the assumption modern **zero-trust** security models reject — any other compromised workload inside the same VPC (a vulnerable dependency in an unrelated service, a misconfigured container with broader network access than intended) can now passively sniff or actively intercept that internal plaintext traffic, including credentials and sensitive payloads, with no TLS protection at all. It's also a compliance problem outright for some regulated data classes (PCI-DSS, HIPAA) that specifically require encryption in transit end-to-end, not just on the public-facing hop. The honest, senior-level answer: it's an acceptable trade-off for genuinely low-sensitivity internal traffic in a tightly-controlled network, and an unacceptable one the moment either the data is sensitive or the internal network's isolation can't be fully guaranteed — which is exactly why re-encrypting internally (mutual TLS between the LB and backends) is increasingly the default recommendation rather than the exception.

zero-trust   mutual TLS internally   compliance requirements

### WHAT THE WEB SOURCES EMPHASIZE

Search sources converge specifically on "beyond 'it's encrypted'" as the explicit framing interviewers use for TLS questions — rewarding the authentication-vs-encryption distinction (6.4) and forward secrecy (6.5) as the two details that separate a memorized answer from a genuinely understood one.

## 07 Infra Networking: LB, CDN, VPC, Firewalls

Where networking theory becomes production cloud architecture — grounded in AWS's own documented terminology for load balancers, NAT gateways, and VPC components.

## 7.1 BASIC COMPARISON Load balancer — Layer 4 vs Layer 7.

|                          | LAYER 4 (NETWORK LB)             | LAYER 7 (APPLICATION LB)                                   |
|--------------------------|----------------------------------|------------------------------------------------------------|
| <b>Operates on</b>       | IP + TCP/UDP port only           | Full HTTP/HTTPS request (path, headers, cookies)           |
| <b>Routing decisions</b> | Cannot see request content       | URL-based routing, header-based routing, WebSocket support |
| <b>Performance</b>       | Higher throughput, lower latency | More overhead, but far more routing flexibility            |
| <b>AWS example</b>       | Network Load Balancer (NLB)      | Application Load Balancer (ALB)                            |

The choice is a direct trade-off between raw performance and routing intelligence: an L4 LB is the right call for pure TCP/UDP traffic at extreme scale (it can't inspect HTTP at all, so it's not a choice — it's a requirement for non-HTTP protocols); an L7 LB is the right call whenever routing needs to depend on the actual request content, such as directing `/api/*` to one backend service and `/static/*` to another from a single entry point.

Layer 4 vs Layer 7 LB

ALB vs NLB

## 7.2 BASIC COMPARISON Reverse proxy vs forward proxy.

A **forward proxy** sits in front of *clients*, forwarding their outbound requests to the Internet on their behalf — the destination server sees the proxy's IP, not the client's, and typically doesn't even know a proxy is involved (used for client anonymity, content filtering, or corporate outbound policy enforcement). A **reverse proxy** sits in front of *servers*, receiving inbound client requests and forwarding them to one or more backend servers on the servers' behalf — the client believes it's talking directly to "the service," with no visibility into how many actual backend instances exist behind it. A load balancer (7.1) is, structurally, a specific kind of reverse proxy — one whose primary job is distributing load across multiple backend instances, though reverse proxies are also used for TLS termination (6.6), caching, and request rewriting independent of load balancing specifically.

forward proxy

reverse proxy

### 7.3 INTERMEDIATE CONCEPT What is a CDN, and how does it reduce latency?

A CDN (Content Delivery Network) is a geographically distributed network of edge servers that cache and serve content from a location physically close to the requesting user, rather than every request traveling all the way back to one origin server. Because network latency is fundamentally bound by the physical distance data must travel (ultimately limited by the speed of light over real fiber routes, plus the round trips of the underlying handshakes), moving the responding server physically closer to the user directly reduces round-trip time in a way no amount of origin-server optimization alone can fix. CDNs commonly use **anycast** routing — the same IP address is announced from many physical edge locations simultaneously, and standard Internet routing (BGP) naturally directs each user's traffic to whichever announcing location is topologically closest, with no explicit client-side logic needed to pick a nearby edge.

edge caching

anycast

### 7.4 INTERMEDIATE CONCEPT What is a VPC, and what are its core components?

A VPC (Virtual Private Cloud — Azure's equivalent is a VNet) is a logically isolated, software-defined private network within a cloud provider, giving an account its own private IP address range and full control over routing, subnetting, and access — the cloud-era equivalent of a physical private network, without owning any physical hardware.

| COMPONENT                      | PURPOSE                                                                                                                                                          |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Subnets</b>                 | Divide the VPC's address range into smaller segments — commonly public (internet-facing) and private (internal-only)                                             |
| <b>Route tables</b>            | Define where traffic from each subnet is directed                                                                                                                |
| <b>Internet Gateway</b>        | Provides the public subnet's actual path to/from the Internet                                                                                                    |
| <b>NAT Gateway</b>             | Lets private-subnet resources reach the Internet outbound (updates, external APIs) without being directly reachable inbound (2.4's SNAT, applied at cloud scale) |
| <b>Security groups / NACLs</b> | Firewall rules controlling allowed traffic (7.5)                                                                                                                 |

VPC

public/private subnet

Internet Gateway

NAT Gateway

## 7.5 INTERMEDIATE SECURITY Stateful vs stateless firewall.

A **stateless** firewall evaluates every packet independently against its rule set, with no memory of prior packets — it must have an explicit rule permitting both the outbound request *and* the corresponding inbound reply, since it has no concept of "this reply belongs to a connection I already approved." A **stateful** firewall tracks active connections, and automatically permits return traffic that's part of a connection it already approved outbound — you only need to write the outbound rule, and legitimate replies are let through without a separate explicit rule, while unsolicited inbound traffic that doesn't match any tracked connection is still blocked by default. AWS's own documented distinction maps directly onto this: **Security Groups** are stateful (the common, default choice for most resources), while **Network ACLs** are stateless and require explicit rules in both directions — a frequent, documented source of confusing "it works one way but not the other" misconfigurations for engineers who assume NACLs behave like security groups.

[stateful firewall](#)[stateless firewall](#)[Security Group vs NACL](#)

## 7.6 ADVANCED COMPARISON

### Load balancing algorithms — round robin, least connections, consistent hashing.

| ALGORITHM                 | HOW IT PICKS A BACKEND                                                                           | BEST FOR                                                                                        |
|---------------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>Round robin</b>        | Cycles through backends in fixed order                                                           | Simple, roughly equal-capacity backends, uniform request cost                                   |
| <b>Least connections</b>  | Sends to whichever backend currently has the fewest active connections                           | Requests with highly variable duration/cost — adapts to real-time load, not just count          |
| <b>Consistent hashing</b> | Hashes a request attribute (e.g. user ID) to consistently route the same key to the same backend | Caching layers, session affinity, minimizing cache-cold reshuffling when backends scale up/down |

Consistent hashing's specific advantage over a naive modulo-based hash: when a backend is added or removed, only a small fraction of keys need to remap to a different backend, rather than nearly all of them — critical for not mass-invalidating a distributed cache every time the backend fleet scales.

[round robin](#)[least connections](#)[consistent hashing](#)

- **Sticky sessions enabled** — if session affinity is configured, the same client is always routed to the same backend for the life of their session; a small number of unusually active users can then concentrate disproportionate load on whichever backend happened to receive them first.
- **Keep-alive connection reuse** — with persistent HTTP connections, a load balancer that only makes its routing decision once (at connection-establishment time) rather than per-request will send every subsequent request on that reused connection to the same backend, even under an algorithm like round robin that assumes fresh, independent routing decisions per request.
- **Algorithm mismatch with actual request cost** — round robin assumes roughly equal request cost; if some requests are far more expensive than others, an equal count of connections per backend can still mean wildly unequal actual load, which is exactly the case least-connections is designed to handle better.
- **Uneven backend capacity** — genuinely different instance sizes/types behind the same LB without weighted routing configured to account for the difference.

The diagnostic order: first check the LB's own configuration (sticky sessions, algorithm) before assuming an application-level bug — this is a very common, purely-configuration root cause that's easy to overlook while debugging the application code instead.

sticky sessions

keep-alive skews routing

**Place TLS termination, a CDN, and a load balancer for a public web service.**

```

Client
| DNS resolves to the CDN's anycast IP (7.3)
v
CDN edge — serves cached static assets directly (images, JS, CSS)
| TLS terminated here for the client-facing hop (6.6)
| (cache miss, or dynamic/uncacheable request) forwarded onward,
| typically re-encrypted for this hop
v
Load Balancer (L7/ALB, 7.1) — TLS terminated again if re-encrypted above
| routes by path/host to the correct backend service
| distributes across healthy instances (7.6)
v
Backend application servers — inside a private VPC subnet (7.4)
| plaintext or internal mTLS depending on the 6.8 risk decision
v
Database / internal services

```

The design logic, tying the whole section together: the CDN handles geographic latency (7.3) and offloads static content entirely; the load balancer handles distributing the remaining dynamic traffic across a scalable backend fleet (7.1, 7.6) and is where application-level routing decisions live; TLS is terminated at the outermost public-facing edge for the strongest client-facing security posture, with the internal-hop encryption decision made deliberately per 6.8's risk framework, not by default in either direction.

CDN → LB → backend architecture

**WHAT THE WEB SOURCES EMPHASIZE**

AWS's own documentation and multiple AWS-focused interview-question sources converge on the same core VPC vocabulary (subnets, route tables, Internet Gateway, NAT Gateway) and the ALB/NLB L7-vs-L4 distinction as the standard cloud-networking interview baseline — used directly rather than a generic, provider-agnostic description.

## 08 Reliability Engineering Patterns

Where networking meets distributed-systems engineering: what a caller should do when a network call fails, and why the naive answer ("just retry") makes real incidents worse.

---

## 8.1 BASIC CONCEPT What is a timeout, and why must every network call have one?

A timeout is a maximum duration a caller will wait for a response before giving up and treating the call as failed. Without one, a caller waiting on a hung or extremely slow downstream dependency will simply wait forever (or until the OS's own default TCP timeout, which can be minutes) — during which time it's typically holding a thread, a connection, or another finite resource, unable to serve any other work. Under real load, this is exactly how one slow downstream dependency cascades into a total outage of everything calling it: callers pile up waiting, exhaust their own thread pools or connection limits, and become unresponsive themselves to their own callers — a timeout is the first, most basic circuit-breaker (8.4) against exactly this cascade, converting an unbounded wait into a bounded, predictable failure the caller can actually react to.

timeout

resource exhaustion cascade

---

## 8.2 BASIC CONCEPT What is idempotency, and why does it matter for safe retries?

Revisiting 5.2's definition in its reliability context: an idempotent operation produces the same end result whether called once or many times. This matters directly for retries because a network call can fail in a genuinely ambiguous way — the request might never have reached the server at all, *or* it might have reached the server, been fully processed, and only the *response* was lost on the way back; from the caller's perspective, both failures look identical (a timeout or connection error). If the operation is idempotent, retrying is always safe regardless of which actually happened — worst case, the same safe operation just runs again with no different outcome. If the operation is *not* idempotent (charging a payment, sending an email), blindly retrying risks doing it twice for real — which is exactly why non-idempotent operations need an explicit idempotency key (8.6) before retries can be considered safe at all.

idempotency

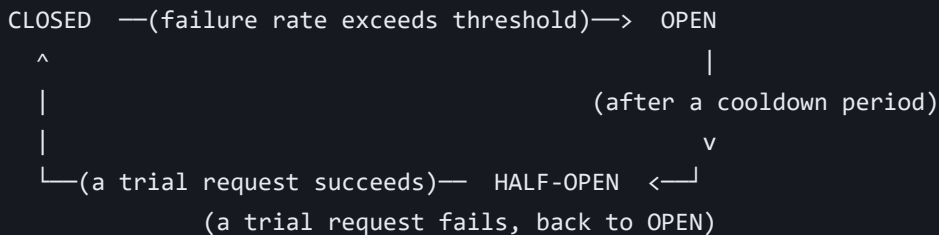
ambiguous failure (request vs response lost)

## Exponential backoff with jitter — what problem does jitter specifically solve?

**Exponential backoff** increases the delay between retry attempts geometrically (e.g. 500ms, 1s, 2s, 4s, 8s) rather than retrying immediately or at a fixed interval — giving a struggling downstream service progressively more room to recover rather than adding to its load at a constant, undiminishing rate. **Jitter** adds randomness to each computed delay, and solves a distinct, separate problem: the **thundering herd**. If a downstream service fails and a thousand callers all independently compute the exact same backoff schedule (500ms, 1s, 2s...), they all retry again at the exact same moments — a synchronized wave of retries that can look, to the recovering service, exactly like another full-scale traffic spike, potentially re-triggering the same failure the backoff was supposed to help recover from. Adding random jitter to each caller's delay spreads those retries out across time instead of them all landing simultaneously, which is what actually gives the downstream service a genuine chance to recover under a smoothed-out, rather than re-synchronized, retry load.

exponential backoff   jitter   thundering herd

## 8.4 INTERMEDIATE RELIABILITY What is a circuit breaker? Walk through its three states.



A circuit breaker wraps calls to a downstream dependency and tracks its recent failure rate. In the **Closed** state (normal operation), requests pass through freely while failures are counted. Once failures cross a configured threshold, the breaker **trips to Open**: for a cooldown period, calls are rejected immediately, locally, without even attempting the network call — this is the key mechanism, protecting both the caller (no more wasted time waiting on a dependency that's currently failing) and the struggling downstream service (no more load added while it's trying to recover). After the cooldown, the breaker moves to **Half-Open**, cautiously allowing a small number of trial requests through: if they succeed, it closes again and resumes normal traffic; if they fail, it immediately trips back to Open for another cooldown period. This is the natural complement to 8.3's backoff — where retries manage a single caller's own request pattern, a circuit breaker manages whether to even attempt calling the dependency at all.

Closed/Open/Half-Open   fail fast

## A downstream service degrades, and naive retries make it worse. Diagnose and fix.

This is a **retry storm**: the downstream service slows down (perhaps just past a normal load threshold), callers' requests start timing out, every caller independently retries, which adds load directly on top of a service that's already struggling — pushing it further into degradation, causing more timeouts, causing more retries, in a self-reinforcing feedback loop that can turn a minor slowdown into a full outage, often worse and longer-lasting than the original problem that triggered it.

The documented, layered fix — each piece addressing a different part of the failure: **(1) Timeouts** (8.1) bound how long any single failing call can hold resources. **(2) Bounded retries with exponential backoff and jitter** (8.3) — capped at a small fixed number of attempts, never unbounded — reduce how much additional load each individual caller's own retrying adds, and smooth out when those retries actually land. **(3) A circuit breaker** (8.4) stops the pattern at its root by having callers stop calling the struggling service altogether once its failure rate crosses a threshold, giving it real breathing room to recover rather than a slightly-reduced but still-mounting load. Research on combining bounded retries with a circuit breaker specifically found this combination meaningfully outperforms either mechanism alone — the three layers are complementary, not redundant, each closing a gap the others leave open.

`retry storm``timeout + backoff + circuit breaker stack`

### REAL INTERVIEW SIGNAL

Search sources specifically frame this as a "resilience patterns" interview staple precisely because it requires combining three separate concepts correctly, rather than reciting any one of them in isolation.

## What is an idempotency key, and how would you implement one for a payment API?

An idempotency key is a unique identifier the *client* generates and attaches to a request, letting the server recognize and safely handle a retried request as "the same operation I already processed," instead of a genuinely new one — directly solving 8.2's problem for operations that aren't naturally idempotent.

```
POST /payments HTTP/1.1
Idempotency-Key: 8f14e45f-ceed-467e-bd2c-a1b2c3d4e5f6
Content-Type: application/json

{"amount": 5000, "currency": "USD", "customer_id": "cus_123"}
```

Server-side implementation: on receiving a request, the server first checks whether it has already seen this exact idempotency key. If not, it processes the payment normally, then stores the key alongside the final result (success or failure) before returning the response. If the same key arrives again — because the client's original request timed out and it retried, per 8.2's ambiguous-failure scenario — the server does *not* process the payment a second time; it simply returns the *stored result* from the first, original attempt. This is precisely what makes it safe to apply the full 8.5 retry stack even to a fundamentally non-idempotent operation like a payment: the retry logic can be generic and blind to what the operation actually does, because the idempotency key is what makes repetition safe at the server, not any cleverness on the client's part.

idempotency key    client-generated, server-deduplicated

## Retrying at the client vs retrying inside a service mesh/proxy — trade-offs?

**Client-side retry logic** (implemented directly in each calling application's code) has full context about the specific call being made — whether it's idempotent, what a sensible timeout is for this specific operation — but means every single service in an organization must correctly implement and maintain its own retry/backoff/circuit-breaker logic, and inconsistencies between teams' implementations are a real, common source of the exact retry-storm problem in 8.5 (one team's overly-aggressive retry config unknowingly hammering a struggling shared dependency).

**Mesh/proxy-level retries** (implemented once, centrally, in infrastructure like a service mesh sidecar or API gateway that every call passes through) guarantee consistent behavior across every service without each team re-implementing it, and can apply organization-wide circuit-breaking with visibility across all callers of a given dependency — but the proxy generally lacks the specific application-level context to know whether a particular request was actually idempotent, and blindly retrying at this layer risks exactly the double-processing 8.2 warns about unless the mesh is specifically configured per-route with that knowledge. The pattern increasingly recommended in practice: use mesh-level retries for the generic, safe case (idempotent reads, connection-level failures) and require explicit, deliberate application-level handling (via 8.6's idempotency keys) for anything that isn't safely retrievable by default.

service mesh retries

centralized vs distributed retry logic

### WHAT THE WEB SOURCES EMPHASIZE

Multiple independent sources converge on the exact same three-layer stack — timeout, bounded retry with backoff+jitter, circuit breaker — as the standard resilience answer, with recent (2026) research specifically quantifying that the combination outperforms any single mechanism, directly grounding 8.5's fix as evidence-based rather than folklore.

## 09 Packet Journey & Debugging Toolkit

The synthesis section — the canonical "what happens when you type a URL" question, and the command-line toolkit every earlier section's debugging answers actually depend on.

**Walk through, end-to-end, what happens when you type a URL and press enter.**

1. **URL parsing** — the browser splits the URL into scheme (https), host, path, and query — deciding which protocol and port to use.
2. **DNS resolution** (Section 4) — browser cache → OS cache → recursive resolver → root/TLD/authoritative chain, ultimately returning an IP address.
3. **TCP handshake** (3.2) — the browser opens a TCP connection to that IP on port 443 (SYN → SYN-ACK → ACK).
4. **TLS handshake** (6.2) — ClientHello/ServerHello, certificate validation (6.3/6.4), and derivation of the symmetric session key — after this, the connection is encrypted.
5. **HTTP request/response** (5.1) — the browser sends a GET request for the page; the server returns a 200 OK with the HTML.
6. **Rendering** — the browser parses the HTML, discovers further resources (CSS, JS, images), and issues additional requests for each — increasingly over the *same* already-open, multiplexed connection if HTTP/2 or HTTP/3 (5.3) is in use, rather than one new connection per asset.
7. **Progressive paint** — the browser builds the DOM/CSSOM and begins painting content as it becomes available, rather than waiting for every last resource to finish loading.

The synthesis skill being tested: naming the specific mechanism at each step (not just "it does DNS stuff") and correctly ordering them — DNS before TCP, TCP before TLS, TLS before the actual HTTP exchange.

end-to-end synthesis

## 9.2 BASIC CONCEPT The core debugging toolkit.

| TOOL                    | WHAT IT TELLS YOU                                                                                         |
|-------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>dig / nslookup</b>   | DNS resolution — does the name resolve, to what, from which server                                        |
| <b>ping</b>             | Basic ICMP reachability and round-trip latency (not TCP-layer proof, per 1.7)                             |
| <b>traceroute / mtr</b> | The router-by-router path to a destination, and where loss/latency is concentrated                        |
| <b>ss</b>               | Current socket state on this machine — listening ports, established connections, retransmit counts        |
| <b>tcpdump</b>          | Raw packet capture — the ground truth when everything else is ambiguous                                   |
| <b>curl -v</b>          | A full, verbose single request — DNS, connect, TLS handshake, and HTTP exchange, each phase visibly timed |

The documented, efficient troubleshooting order (per current SRE-focused sources): confirm the interface is up, resolve the name, prove basic reachability, trace the path, test the specific port, check the local listener, and only then reach for a full packet capture — cheapest, fastest checks first, packet-level inspection last.

dig ss tcpdump curl -v

## 9.3 INTERMEDIATE DEBUG DNS resolves fine, but curl times out. Next steps?

```
$ dig example.com +short
203.0.113.10                                # resolves fine

$ curl -v https://example.com
* Trying 203.0.113.10:443...
* connect() timed out!                      # hangs here – DNS is not the problem
```

Per 4.5's logic, DNS is confirmed working — the fault is downstream of resolution, at the TCP connect step specifically (visible directly in `curl -v`'s phased output). Next: ping the resolved IP directly — if that also fails, it's a routing/reachability problem (loop back to 1.6's layer walk); if ping *succeeds* but the TCP connect still hangs, per 1.7 that's the specific signature of a firewall/security-group blocking port 443 specifically while permitting ICMP — check security group/firewall rules for that exact port next, and confirm with `nc -zv 203.0.113.10 443` to isolate the TCP-connect test from HTTP/TLS entirely.

curl -v phased output connect() timeout isolates the layer

## Intermittent retransmissions and latency spikes on one path. Localize it.

```
$ mtr -rw example.com
HOST: laptop
  Loss%  Snt  Last  Avg  Best  Wrst  StDev
1. gateway      0.0%  50    1.2  1.1   0.9   2.0   0.2
2. isp-router-3.net 0.0%  50    8.4  8.1   7.2  12.1   1.1
3. isp-core-7.net  12.0%  50   45.3 41.2  15.4  88.0  18.3 ← loss starts here
4. transit-provider.net 11.8%  50   44.9 40.8  14.9  87.5  18.1 ← and persists downst
5. destination  11.5%  50   45.1 41.0  15.1  87.9  18.0
```

`mtr` (a continuous, statistics-gathering traceroute) is the tool of choice specifically because it reveals loss *at a specific hop* over time, distinguishing genuine path congestion from a one-off blip — loss that first appears at hop 3 and persists through every hop after it indicates the problem is at or immediately after hop 3, not further downstream (hops past the fault show the same loss simply because the earlier drop already happened). This localizes the issue to a specific network segment, typically outside your own control at that point (an ISP or transit-provider link) — the actionable output of this diagnosis is usually "here's the specific hop and provider to report this to," not something fixable from the endpoint alone.

`mtr``loss-at-a-specific-hop`

## Ping succeeds, but the application still can't connect. Every possible reason.

This directly compounds 1.7's core gotcha into a full diagnostic checklist:

- **Firewall/security group blocks the specific TCP port** while permitting ICMP — by far the most common cause (9.3's exact scenario).
- **Nothing is actually listening on that port** on the target host — the application crashed, isn't started, or is bound to a different port/interface than expected (verifiable directly via `ss -tlnp` on the target host, not the client).
- **The application is bound to 127.0.0.1 (localhost) instead of 0.0.0.0** — it's genuinely running and healthy, but only accepts connections originating from the same machine, so any remote connection attempt is refused regardless of network-layer reachability.
- **A load balancer or proxy in front of the target** has its own separate health check marking the backend unhealthy and refusing to route to it, even though the underlying host itself responds fine to ping and even to a direct port check.
- **TLS/application-layer rejection** (a certificate error, an authentication failure) — the TCP connection itself fully succeeds, but the layer above it fails, which can be misreported by a user simply as "can't connect" without the layer distinction being clear.

The unifying lesson across all five: ping only ever tests the Network layer (ICMP) — every one of these five failures lives at the Transport layer or above, entirely invisible to ping by design, not by coincidence.

127.0.0.1 vs 0.0.0.0 binding

LB health check separate from host reachability

## Why can a link have high throughput and still feel slow? What is bufferbloat?

**Latency** (how long one round trip takes) and **throughput** (how much data can move per second, once flowing) are genuinely independent measurements — a satellite link can have enormous throughput but still feel sluggish for interactive use because every single round trip takes hundreds of milliseconds regardless of available bandwidth; a page load composed of many small, sequential, round-trip-bound requests (DNS, TCP handshake, TLS handshake, then the actual request) is dominated by latency far more than by raw bandwidth, which is exactly why reducing round trips (QUIC's 0-1 RTT handshake, HTTP/2/3's multiplexing avoiding new connections per asset) matters more for perceived speed than adding more bandwidth once a reasonable baseline is already available.

**Bufferbloat** is a specific, well-documented pathology that makes this worse: routers and network equipment along a path often have oversized packet buffers, intended to smooth out momentary bursts — but under sustained congestion, instead of dropping excess packets promptly (which is the exact signal TCP's congestion control, Section 3.6, needs to correctly back off), those oversized buffers instead queue the excess packets for a long time before finally delivering (or eventually dropping) them. The link's raw throughput capacity is entirely unaffected, but every packet now sits in a deep queue for a long time before being delivered — latency for every flow sharing that link balloons even though nothing is technically being dropped or "failing," which is precisely why a large download running in the background can make an unrelated video call on the same connection feel terrible, despite the link's advertised bandwidth being more than sufficient for both.

[latency vs throughput](#)[bufferbloat](#)

## An API works from your laptop but times out from a container in the cloud. Checklist?

1. **DNS resolution inside the container** — the container's DNS configuration may differ entirely from the host's (a common issue in containerized environments with their own embedded DNS/service-discovery layer); *dig* run *inside the container itself*, not just from the host, is the first real check.
2. **Security groups / network policies** — the container's specific network context (a Kubernetes namespace, a specific subnet) may be governed by rules that differ from what the developer's laptop or a general-purpose bastion host is subject to; confirm the actual egress rule applies to this specific workload's identity/subnet, not just "the VPC" generally.
3. **MTU mismatch inside an overlay network**
4. **Outbound NAT/SNAT port exhaustion** — at genuine cloud scale, many containers sharing one NAT Gateway's limited pool of external ports (2.4/7.4) can hit a hard connection limit under load, in a way a single developer's laptop making one request would never encounter or reveal.
5. **Egress proxy requirements** — some cloud network architectures require all outbound traffic to route through an explicit egress proxy that a laptop's more permissive default networking wouldn't have needed at all.

The unifying theme across all five: "works on my laptop" is specifically dangerous advice here, because a developer's local network context is systematically different from a production container's — the fix is always testing *from inside the actual failing environment*, not reasoning by analogy from a superficially similar one.

overlay network MTU

NAT port exhaustion at scale

## Synthesis: curl hangs after "Connected," and tcpdump shows a SYN with no SYN-ACK. Diagnose.

```
$ curl -v https://example.com
* Trying 203.0.113.10:443...
* Connected to example.com (203.0.113.10) port 443
* (hangs here indefinitely – no further output)

$ tcpdump -i eth0 host 203.0.113.10 and port 443
12:00:01.001 IP client.51234 > 203.0.113.10.443: Flags [S], seq 1000...
    (no SYN-ACK ever arrives in the capture)
```

This is a precise, gradable puzzle rather than a vague "it's slow": `curl` reporting **"Connected"** is actually a slightly misleading milestone — it fires once the DNS lookup and socket setup begin, not once the TCP handshake has genuinely completed (a common misreading of `curl`'s own phased output that itself is worth naming explicitly in an answer). The `tcpdump` capture is the real ground truth: a SYN was sent, and per 3.2, a healthy handshake requires a SYN-ACK in response — the complete absence of one, with the client simply never receiving anything back, means the destination is either not receiving the SYN at all (dropped silently somewhere along the path — a firewall configured to silently drop rather than actively reject, exactly the behavior 9.5's first bullet describes) or is receiving it but choosing not to respond (also typically firewall behavior, or a completely overloaded target that can't service its accept queue). The diagnostic conclusion: this is *not* a DNS problem (name clearly resolved, since a connection attempt to a real IP was made at all) and *not* a TLS problem (the handshake never even reached that layer) — it's a Network/Transport-layer connectivity problem, most likely a silent firewall drop, and the next concrete step is checking security group/firewall rules for port 443 specifically from the client's actual source IP or subnet, exactly as 9.3's checklist prescribes.

`curl "Connected" ≠ handshake complete`

`silent SYN drop = firewall, not DNS/TLS`

### WHAT INTERVIEWERS LISTEN FOR

Correctly ruling out DNS and TLS as candidates — and explaining *why* the evidence rules them out, not just asserting it — is what separates a strong synthesis answer from a guess that happens to land on the right layer.

#### WHAT THE WEB SOURCES EMPHASIZE

The GitHub "what-happens-when" project and multiple senior-engineer interview breakdowns converge on the same ordered flow (DNS → TCP → TLS → HTTP → render) as the expected answer structure, while SRE-focused sources independently converge on the exact same troubleshooting command sequence (interface → dig → ping → mtr → port test → ss → tcpdump) used to structure Section 9's debugging progression.

## Sources consulted

Compiled from current authoritative RFCs, cloud provider documentation, GATE archives, and aggregated practitioner/interview-report sources as of September 2026. Where this guide says a question is "commonly asked" or "frequently reported," that reflects language or data found directly in the cited source, not an invented popularity claim.

[RFC 9293 — Transmission Control Protocol \(TCP\), the current standard](#)

[RFC 5681 — TCP Congestion Control](#)

[RFC 2923 — TCP Problems with Path MTU Discovery](#)

[Cloudflare Blog — HTTP/3: The Past, The Present, and The Future](#)

[Cloudflare Blog — A Last Call for QUIC](#)

[AWS Whitepaper — NAT Gateway and Gateway Load Balancer with EC2](#)

[GeeksforGeeks — What Happens When We Type a URL](#)

[GitHub — what-happens-when \(the canonical interview-question writeup\)](#)

[GeeksforGeeks — Computer Networks for GATE CSE](#)

[Linux Networking Interview Questions Every SRE Should Know](#)

[SamNet — Networking Commands Cheat Sheet \(ip, ss, dig, tcpdump\)](#)

[GeeksforGeeks — Microservices Resilience Patterns](#)

[Cybersecurity Interview Questions — TLS/SSL, MITM, Forward Secrecy](#)